

TP previo Parcial 2 DDS 2026 - Curso 3K3

Campo	Valor
Materia	Desarrollo de Software (DDS)
Año	2026
Instancia de evaluación	TP previo - Parcial 2
Documento	Enunciado de trabajo práctico
Curso	3K3

TP previo Parcial 2 DDS 2026 - Curso 3K3

Introducción

Desde la publicación de la tarea hasta el sábado 13/06 a las 23:55 hs. Cada grupo de entre 2 y 5 alumnos va a desarrollar una aplicación full stack para controlar solicitudes de equipamiento. El sistema debe administrar equipos, disponibilidad, préstamos, autorizaciones y devoluciones, contemplando que no todos los usuarios pueden aprobar ni modificar cualquier solicitud. El objetivo es construir una solución que tenga reglas de dominio y trazabilidad, no un listado simple de equipos. Adicionalmente cada alumno deberá subir el trabajo realizado a su portafolio.

Desde el 14/6 10 hs hasta el lunes 15/6 a las 23:55 otro grupo deberá realizar una revisión / corrección entre pares (revisión entre grupos) y entregar la devolución grupal.

Estas dos tareas son esenciales y previas al segundo parcial ya que habrá 2 preguntas a desarrollar relativas el tema desarrollado.

Este práctico integra los contenidos principales del segundo parcial: Express Router, middlewares, manejo centralizado de errores, CORS, autenticación y autorización con JWT, pruebas de API con Jest y Supertest, React con Vite, componentes, estado, efectos, React Router, formularios, Axios, validaciones y manejo de errores de API.

Como el trabajo se resuelve en grupo y con una semana de desarrollo, se espera una solución de complejidad media-alta: reglas de negocio reales, estados, permisos, historial, filtros avanzados, persistencia, datos semilla, pruebas automatizadas y una interfaz navegable que permita operar el sistema completo.

Alcance mínimo de resolución grupal

La entrega debe evidenciar que hubo división de responsabilidades y posterior integración. Como mínimo, el sistema debe incluir:

- un módulo de autenticación completo, con registro, login, JWT, roles y persistencia de sesión en frontend;
- un módulo principal del dominio con listado, filtros, detalle, alta, edición y acciones de estado;
- un módulo de administración o resumen que muestre información agregada del dominio;
- validaciones de negocio implementadas en servicios del backend, no solamente en formularios;
- al menos dos flujos end-to-end funcionando desde la interfaz hasta la persistencia;
- pruebas automatizadas que cubran casos exitosos, errores de validación, errores de permisos y reglas específicas del dominio;
- documentación suficiente para que otro grupo pueda ejecutar, probar y explicar la solución.

No se considera suficiente una entrega que solo tenga pantallas sin backend real, endpoints sin frontend integrado, datos hardcodeados en componentes, o reglas de negocio aplicadas únicamente de forma visual.

¿Qué tienen que construir?

Desarrollar una aplicación web full stack para controlar solicitudes de equipamiento. La aplicación debe permitir pedir equipos, validar si están disponibles y aplicar autorización cuando el equipo lo requiera. No es un CRUD genérico: una solicitud debe depender del estado del equipo y del rol del usuario que intenta aprobarla.

En el parcial presencial cada estudiante responderá preguntas escritas sobre la solución entregada por su grupo. Deben poder explicar rutas, componentes, servicios, middlewares, validaciones, JWT, permisos, pruebas y errores manejados.

Dominio obligatorio

El sistema trabaja sobre el dominio de control de equipamiento. El recurso principal que se administra es la Solicitud: un pedido de préstamo de un equipo durante un período determinado. Cada solicitud debe estar asociada obligatoriamente a un Equipo, que representa un bien inventariado con categoría, ubicación, estado y posible requerimiento de autorización.

La regla central del dominio es que una solicitud solo puede aprobarse si el equipo existe, está disponible, no tiene otro préstamo aprobado superpuesto y, cuando el equipo requiere autorización, la aprobación la realiza un usuario con rol habilitado.

Entidades mínimas obligatorias

El sistema debe modelar y persistir al menos 4 entidades. Si usan archivo JSON, SQLite, Sequelize u otra alternativa, igual deben respetar esta separación lógica:

Entidad	Para qué existe	Campos mínimos	Relaciones principales
usuarios	Representa solicitantes, encargados y administradores que operan el préstamo.	id, nombre, email, passwordHash, rol, activo	Un usuario puede crear solicitudes, autorizarlas y generar historial.
equipos	Representa bienes inventariados que pueden solicitarse.	id, codigoInventario, nombre, categoria, estado, ubicacion, requiereAutorizacion	Un equipo puede estar asociado a muchas solicitudes.
solicitudes	Representa el pedido de préstamo de un equipo por un período.	id, equipoId, usuarioId, fechaRetiro, fechaDevolucion, motivo, estado, autorizadoPor	Pertenece a un equipo y a un usuario solicitante; puede tener usuario autorizante.
historial_solicitudes	Registra auditoría de cambios sobre solicitudes.	id, solicitudId, usuarioId, accion, fechaHora, valorAnterior, valorNuevo	Pertenece a una solicitud y al usuario que hizo la acción.

Campos mínimos por entidad

Los campos pueden adaptarse al motor de persistencia elegido, pero la información y las relaciones deben estar presentes.

Usuario

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí	Identificador único del usuario.	1, usr-001
nombre	string	Sí	Nombre visible del usuario.	Carla Ruiz
email	string	Sí	Credencial de acceso y dato de contacto. Debe ser único.	carla@dds.com
passwordHash	string	Sí	Contraseña almacenada de forma segura o hash simulado	bcrypt_hash

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
			documentado.	
rol	string	Sí	Define permisos dentro del sistema.	usuario, encargado, admin
activo	boolean	Sí	Permite bloquear accesos sin borrar el usuario.	true, false

Equipo

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí	Identificador único del equipo.	eq-001
codigoInventario	string	Sí	Código institucional o patrimonial. Debe ser único.	INV-2026-014
nombre	string	Sí	Nombre reconocible del equipo.	Notebook Dell 14
categoria	string	Sí	Tipo de equipo.	notebook, proyector, cámara, kit de red
estado	string	Sí	Disponibilidad operativa.	disponible, prestado, mantenimiento, baja
ubicacion	string	Sí	Lugar donde se encuentra el equipo.	Laboratorio A5
requiereAutorizacion	boolean	Sí	Indica si necesita aprobación especial.	true, false

Solicitud

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí	Identificador único de la solicitud.	sol-1001
equipoId	numero/string	Sí	Equipo	eq-001

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
			solicitado. Debe existir en equipos.	
usuarioId	numero/string	Sí	Usuario que pide el préstamo.	usr-001
fechaRetiro	date/string	Sí	Fecha desde la que se necesita el equipo.	2026-06-18
fechaDevolucion	date/string	Sí	Fecha prevista de devolución. Debe ser posterior al retiro.	2026-06-20
motivo	string	Sí	Justificación del préstamo.	Práctica de laboratorio
estado	string	Sí	Estado actual del flujo.	pendiente, aprobada, rechazada, cancelada, devuelta
autorizadoPor	numero/string	No	Usuario que aprobó la solicitud, si corresponde.	usr-admin

HistorialSolicitud

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	numero/string	Sí	Identificador único del registro de auditoría.	hist-001
solicitudId	numero/string	Sí	Solicitud afectada por el cambio.	sol-1001
usuarioId	numero/string	Sí	Usuario que realizó la acción.	usr-admin
accion	string	Sí	Operación realizada sobre la solicitud.	creacion, edicion, aprobacion, rechazo, cancelacion, devolucion
fechaHora	datetime/string	Sí	Momento exacto de la acción.	2026-06-08T11:15:00
valorAnterior	object/string	No	Datos relevantes	{ "estado": "pendiente" }

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
			antes del cambio.	
valorNuevo	object/string	No	Datos relevantes después del cambio.	{ "estado": "aprobada" }

Reglas funcionales específicas

La aplicación debe resolver estos casos:

1. Registrar usuario e iniciar sesión.
2. Obtener un JWT al iniciar sesión correctamente.
3. Listar solicitudes con filtros combinables por estado, equipoId, categoría y rango de fechas.
4. Ver el detalle de una solicitud desde una ruta dinámica.
5. Crear una solicitud seleccionando un equipo existente.
6. Editar fechas o motivo mientras la solicitud esté pendiente.
7. Cancelar una solicitud propia si no fue devuelta.
8. Aprobar o rechazar solicitudes desde rol autorizado.
9. Rechazar la solicitud si fechaRetiro es posterior o igual a fechaDevolucion.
10. Rechazar la solicitud si el equipo no está disponible o si ya tiene otra solicitud aprobada para fechas superpuestas.

Complejidad adicional obligatoria

El grupo debe implementar también:

11. Flujo completo de estados: pendiente -> aprobada o rechazada; pendiente o aprobada -> cancelada; aprobada -> devuelta. No permitir devolver solicitudes no aprobadas.
12. Búsqueda paginada y ordenable: aceptar page, limit, sortBy y order en el listado de solicitudes.
13. Vista resumen para administración con equipos disponibles por categoría, solicitudes pendientes de aprobación y préstamos vencidos.
14. Historial de cambios de cada solicitud: guardar fecha, usuario, acción y valores modificados cuando se aprueba, rechaza, cancela, devuelve o edita.
15. Cálculo de solicitudes vencidas cuando fechaDevolucion sea anterior a la fecha actual y el estado siga aprobada.
16. Cambio automático o validado del estado del equipo cuando una solicitud se aprueba o se devuelve.
17. Semilla de datos inicial con al menos 8 equipos, 2 usuarios comunes, 1 encargado/admin y 12 solicitudes en distintos estados.
18. Mensajes de error distintos para equipo inexistente, equipo no disponible, fechas inválidas, superposición y falta de permisos.

Aclaraciones para resolver el alcance

Estas aclaraciones responden dudas esperables al momento de implementar:

- ¿Cuándo un equipo está disponible? Debe tener estado disponible y no debe tener una solicitud aprobada superpuesta con el período solicitado.
- ¿Qué solicitudes bloquean disponibilidad? Solo las solicitudes aprobadas. Las pendientes pueden mostrarse como advertencia, pero no bloquean salvo que el grupo decida documentarlo y aplicarlo consistentemente.
- ¿Qué pasa con equipos que requieren autorización? Un usuario común puede solicitarlos, pero solo admin o encargado puede aprobar esas solicitudes.
- ¿Cuándo cambia el estado del equipo? Al aprobar una solicitud puede pasar a prestado; al devolverla debe volver a disponible, salvo que se documente otro criterio.
- ¿Qué debe mostrar el resumen? Como mínimo: equipos disponibles por categoría, solicitudes pendientes, equipos prestados y préstamos vencidos.
- ¿Dónde se valida la disponibilidad? En el servicio del backend. No alcanza con deshabilitar equipos desde la interfaz.

Roles y permisos

Deben existir al menos estos roles:

- usuario: puede crear solicitudes, ver sus solicitudes y cancelar las propias.
- admin o encargado: puede ver todas las solicitudes, aprobar, rechazar y marcar devolución.

Al menos una ruta de escritura debe devolver:

- 401 si no se envía JWT.
- 403 si el usuario está autenticado pero no tiene permiso para la acción.

Backend esperado

Implementar backend con Node.js y Express.

Rutas mínimas:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/equipos
- GET /api/solicitudes?estado=&equipoId=&categoria=&desde=&hasta=
- GET /api/solicitudes/resumen
- GET /api/solicitudes/:id

- GET /api/solicitudes/:id/historial
- POST /api/solicitudes
- PUT /api/solicitudes/:id
- PATCH /api/solicitudes/:id/cancelar
- PATCH /api/solicitudes/:id/aprobar
- PATCH /api/solicitudes/:id/rechazar
- PATCH /api/solicitudes/:id/devolver

Estructura mínima:

- routes/solicitudes.routes.js con express.Router()
- controlador de solicitudes
- servicio de solicitudes con reglas de disponibilidad y autorización
- middleware de autenticación JWT
- middleware de autorización por rol o propietario
- middleware de validación de entrada
- middleware de manejo de errores con firma (err, req, res, next)
- persistencia en archivo JSON, SQLite, Sequelize u otra solución que conserve datos al reiniciar el backend
- carga de datos semilla para probar el dominio sin cargar todo manualmente

Usar status HTTP coherentes: 200, 201, 400, 401, 403, 404 y 500. Las respuestas de error deben tener JSON claro, por ejemplo { "error": "El equipo no está disponible para el período solicitado" }.

Frontend esperado

Implementar frontend con React y Vite.

Pantallas mínimas:

19. Login y registro.
20. Listado de solicitudes con filtros por estado, categoría y fechas.
21. Detalle de solicitud en una ruta como /solicitudes/:id.
22. Pantalla transaccional de alta/edición de solicitud, donde se selecciona equipo, período de préstamo y motivo, y se confirma la operación contra la API.
23. Acciones visibles para cancelar, aprobar, rechazar o devolver según rol.
24. Panel resumen para administradores o encargados.
25. Historial visible dentro del detalle de una solicitud.
26. Ruta comodín para página no encontrada.

La capa de servicios debe usar Axios:

- instancia con baseURL
- params para filtros
- body en POST, PUT o PATCH
- header Authorization: Bearer <token> en acciones protegidas
- manejo visible de errores de validación, autenticación, autorización y recurso inexistente
- componentes separados para tabla/listado, filtros, formulario, detalle, acciones por rol y resumen administrativo
- estados de carga, vacío, error y éxito para las operaciones principales

Integración y calidad esperada

La resolución debe mostrar trabajo de equipo y no una acumulación de archivos sueltos. Se espera:

- contexto, hook o mecanismo equivalente para conservar usuario autenticado, token y rol en frontend;
- rutas protegidas en frontend y backend, no solo botones ocultos;
- validaciones repetidas en frontend para experiencia de usuario y en backend como fuente de verdad;
- servicios Axios separados por recurso, sin llamadas HTTP mezcladas dentro de todos los componentes;
- contraseñas hasheadas o, si usan usuarios semilla simplificados, aclaración explícita de la limitación en el README;
- payload del JWT sin contraseñas ni datos sensibles;
- manejo de errores centralizado en backend y mensajes comprensibles en frontend;
- decisión documentada sobre persistencia, estructura de carpetas y división de responsabilidades del grupo.

Testing mínimo

El backend debe incluir pruebas con Jest y Supertest para:

1. Login correcto e inválido.
2. Listado de solicitudes con y sin filtros.
3. Detalle de solicitud existente e inexistente.
4. Creación válida de una solicitud.
5. Creación inválida por fechas inconsistentes.
6. Creación inválida por equipo no disponible o superposición.
7. Acceso sin JWT a una ruta protegida.
8. Acceso con JWT de usuario a una acción solo permitida para admin o encargado.
9. Devolución inválida de una solicitud no aprobada.
10. Transición de estado no permitida, por ejemplo aprobar una solicitud cancelada.

Las pruebas deben verificar status HTTP y cuerpo JSON relevante.

Documentación obligatoria

La entrega debe incluir un README.md con:

- instrucciones para ejecutar backend y frontend
- usuario admin o encargado y usuario usuario de prueba
- listado de endpoints principales
- listado de rutas frontend
- explicación de disponibilidad del equipo y autorización de préstamos
- explicación de JWT, roles y permisos
- comando para ejecutar pruebas
- limitaciones conocidas

Preguntas frecuentes y criterios de corrección

Sobre alcance y evaluación

- El TP es condición de entrega y también será la base de preguntas abiertas del Parcial 2. La nota del parcial surge del examen, pero una entrega incompleta o no explicable afecta directamente las respuestas escritas.
- En el parcial se preguntará sobre el código real entregado por el grupo: rutas, servicios, componentes, validaciones, JWT, permisos, pruebas y decisiones.
- Se aceptan soluciones equivalentes si respetan entidades mínimas, reglas del dominio, rutas, pantallas, pruebas y criterios de seguridad. Cualquier cambio importante debe estar justificado en el README.
- La persistencia puede ser archivo JSON, SQLite, Sequelize u otra alternativa, siempre que conserve datos al reiniciar y respete las entidades del enunciado.
- Debe haber registro/login real desde frontend. Los usuarios semilla son obligatorios para probar rápido, pero no reemplazan el flujo de autenticación.
- El historial debe existir en backend y verse desde frontend, al menos en el detalle del recurso principal.
- La pantalla transaccional puede ser una sola pantalla para alta/edición o dos pantallas separadas, pero debe confirmar la operación contra la API y mostrar errores.
- El resumen administrativo debe estar protegido para roles administrativos.
- Filtros, paginación y ordenamiento deben resolverse en backend. El frontend solo envía params y muestra resultados.

- Documentar una limitación no reemplaza un requisito mínimo. Sirve para explicar decisiones o faltantes menores, no para omitir reglas centrales.

Sobre backend

- Deben usar `express.Router()` en archivos separados para el recurso principal.
- El middleware de errores con firma `(err, req, res, next)` debe quedar después de las rutas.
- Los errores de validación deben responder con status coherente, normalmente 400, y JSON claro, por ejemplo `{ "error": "mensaje" }`.
- La autorización debe validar rol y, cuando corresponda, propiedad del recurso. No alcanza con que el JWT exista.
- El JWT no debe incluir contraseñas ni datos sensibles.
- Las contraseñas deben guardarse hasheadas; si se usa una simplificación por tiempo, debe estar documentada y no exponer contraseñas en respuestas.
- Las reglas de negocio deben estar en servicios del backend, no solamente en controladores ni formularios.
- Deben estar protegidas las acciones de escritura y cualquier lectura privada o administrativa.

Sobre frontend

- React Router debe incluir ruta comodín `*`.
- Las pantallas de detalle deben leer parámetros con `useParams`.
- Pueden usar React Hook Form o formularios controlados con `useState`, pero las validaciones deben verse en pantalla.
- Los errores de API deben mostrarse de forma comprensible cerca de la acción que falló o en un área visible de la pantalla.
- Axios debe estar en una capa de servicios separada. No mezclar llamadas HTTP en todos los componentes.
- El JWT puede enviarse con interceptores o agregando headers en servicios, pero siempre como `Authorization: Bearer <token>`.
- Proteger una ruta en frontend significa impedir acceso visual/navegable si no hay usuario autenticado o rol suficiente; esto no reemplaza la protección del backend.

Sobre testing

- Se exigen pruebas de backend con Jest y Supertest. Tests de frontend son opcionales.
- Cada test debe validar status HTTP y cuerpo JSON relevante.

- Deben probar login correcto, login inválido, acceso sin token y acceso con rol insuficiente.
- Deben probar reglas específicas del dominio, no solo endpoints felices.
- Los datos de prueba deben ser previsibles. Si un test modifica datos, deben resetearlos o aislarlos para no afectar otros tests.

Entrega

Subir a Moodle el repositorio o archivo comprimido en el portafolio del alumno y en la entrega grupal. No incluir carpetas node_modules

Si deben incluir package.json, código fuente, tests y documentación.